



Review: Output from basic SIS method

- In the example of the basic SIS method, we found that:
 - The particle filter started off tracking well.
 - Performance rapidly deteriorated (especially performance of the confidence bounds).
 - Most of the particle weights become really small.
 - Increasing the number of particles helps only a little.
- If only a few of the weights are large, only a few trajectories contribute to the state estimate:

$$\hat{x}_k = \sum_{i=1}^{N_p} w_k^{(i)} x_k^{(i)},$$

meaning that we are wasting a lot of CPU resources.



Why are the weights collapsing?

- Why did this happen?
 - Recall: In general particles are entire state trajectories $\mathbb{X}_k^{(i)}$.
 - We are drawing samples (randomly) from a high-dimensional space.
 - Difficult to draw probable state trajectories.
 - In the joint particle filter, the dimension of the particles grows linearly with time.
 - It becomes exponentially difficult to sample efficiently.
 - This is known as the degeneracy problem.
 - We observed this problem even when we implemented a marginal particle filter that drew random samples of $x_k^{(i)}$ instead of $\mathbb{X}_k^{(i)}$.
- The problem is manifest as $w_k^{(i)} \rightarrow 0$ for the majority of the particles as k increases.



Defining an effective sample size

- We need an easily computable and meaningful diagnostic.
- We introduce the effective sample size, defined as:

$$N_{pe,k} = \frac{1}{\sum_{i=1}^{N_p} (w_k^{(i)})^2}.$$

- This is not a derivation, but a useful definition.
- Note that it makes sense for the boundary cases:
 - If all weights are equal, $w_k^{(i)} = 1/N_p$ and:

$$N_{pe,k} = \frac{1}{\sum_{i=1}^{N_p} (1/N_p)^2} = \frac{1}{N_p(1/N_p)^2} = N_p.$$

- If all weights but one are zero, we have:

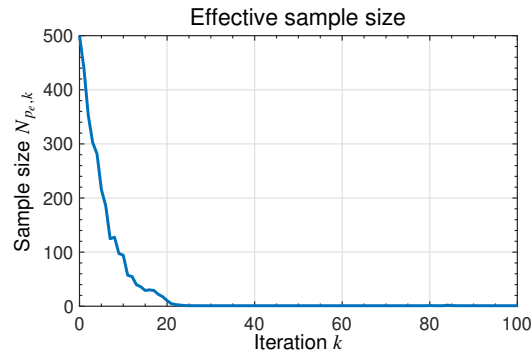
$$N_{pe,k} = \frac{1}{1} = 1.$$



Applying effective sample size to our example

- We apply this metric to the car-tracking simulation example.
- The figure shows the computation of $N_{pe,k}$.

- As expected from its definition, we confirm that $N_{pe,k}$ is bounded and $1 \leq N_{pe,k} \leq N_p$.
- The simulation starts with $N_{pe,0} = N_p$, as desired.
- But, then $N_{pe,k}$ quickly decreases.
- We are essentially calculating the state from a single trajectory after 20 or so samples.



Addressing the problem

- We desire to increase the number of effective particles.
- One idea is to delete particles having low importance weights and replace them with particles having high importance weights.
- We resample from our set $\{x_k^{(i)}, w_k^{(i)}\}$ with probability $w_k^{(i)}$ and with replacement, whenever $N_{pe,k}$ drops below some threshold (perhaps some percentage of N_p).
 - This will replicate high-probability particles, but assuming that our problem has process noise these particles will not remain the same after just one iteration.
 - If there is no process noise, we will need to perturb the replicates slightly.
- Can resample as many times as we like (the number of particles does not need to remain constant), but typically we draw as many samples as we started with.
- The new particles are drawn with a probability given by the importance weights, so we can assign uniform weights $1/N_p$ to the resulting IID samples.



SIS algorithm, with resampling

$$\{x_k^{(i)}, w_k^{(i)}\}_{i=1}^{N_p} = \text{SIS with resampling} \left(\{x_{k-1}^{(i)}, w_{k-1}^{(i)}\}_{i=1}^{N_p}, z_k \right)$$

For $i = 1 : N_p$,

Draw new particle value $x_k^{(i)} \sim q(x_k | x_{k-1}^{(i)}, z_k)$ — randomly!

Calculate unnormalized weight for this particle:

$$\tilde{w}_k^{(i)} = \frac{f(z_k | x_k^{(i)}) f(x_k^{(i)} | x_{k-1}^{(i)})}{q(x_k^{(i)} | x_{k-1}^{(i)}, z_k)} w_{k-1}^{(i)}.$$

End for loop

Normalize the weights for all particles: $w_k^{(i)} = \frac{\tilde{w}_k^{(i)}}{\sum_{i=1}^{N_p} \tilde{w}_k^{(i)}}.$

Compute number of effective particles, $N_{pe,k} = \frac{1}{\sum_{i=1}^{N_p} (w_k^{(i)})^2}.$

If $N_{pe,k} < \text{some preset threshold}$, then resample (see next slide)

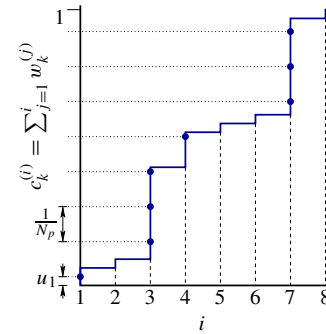


A simple algorithm to perform resampling

- A systematic resampling method is as follows:
 - Let c be the cumulative summation of weights

$$c_k^{(i)} = \sum_{j=1}^i w_k^{(j)}.$$

- Make a single draw $u_1 \sim \mathcal{U}(0, N_p^{-1})$ and set $i = 1$.
- For each $j = 1 : N_p$,
 - $u_j = u_1 + (j - 1)/N_p$.
 - While $u_j > c_k^{(i)}$, $i = i + 1$.
 - Assign new $x_k^{(j)} = \text{old } x_k^{(i)}$, $w_k^{(j)} = 1/N_p$.



- In the illustration, particle one is chosen once, particle two is not chosen, particle three is chosen thrice, and so forth.
- This way some particles are replicated and other particles are dropped.



Summary

- Particle filters propagate trajectories by drawing random values to represent possible future states of the system.
- Randomly drawing values is unlikely to result in highly probable future states, and so the importance weights of most particles tend to collapse to zero over time.
- Therefore, the output of the particle filter depends on only one or a few particles.
- We have defined a new metric, the “effective number of particles.”
- When this metric falls below some specified value, we resample particles from the existing population.
 - This tends to replicate more probable particles and delete less probable particles.
- You have learned one resampling algorithm; in the next lesson you will learn how to implement this algorithm in Octave and will see some results.